

CMPT 276 - Phase 3 Report

Project Group 14

Joshua Kim, Tim Yeh, Rodrigo Anasco, Sarah Harb

The purpose of this report is to document the testing phase of our project, detailing the tests performed, the resulting changes to our code, and the improvements made to our game. Our focus during this phase was to resolve existing bugs, ensure the quality of our code, and achieve high test coverage. Through rigorous testing, we aimed to enhance the overall functionality and reliability of the game, ultimately delivering a better experience for users.

Previous Bugs

Before initiating the testing phase, we identified several critical bugs and areas for improvement in our game. These issues, outlined below, affected both the gameplay experience and the functionality of key features:

1. Menu and Game Synchronization: While the game was running, the key handler for the main menu continued to operate in the background. This led to scenarios where players could accidentally exit the game because the menu was still "active" in parallel with the game.
2. Collision Detection Issues:
 - Rats Walking Through Walls: Rats sometimes ignored walls, disrupting the intended pathfinding logic. Additionally, they could get stuck if the shortest path was blocked instead of recalculating the next best route.
 - Doug Stuck Near Walls: When Doug moved too close to a wall, he occasionally became stuck, suggesting a need for boundary adjustment around barriers.
3. Gameplay Design Improvements:
 - Simplifying the maze by removing some bushes to ensure rats could navigate to Doug more easily.
 - Increasing the movement speed of rats for a more challenging and engaging gameplay experience.
 - Missing Instructions: The instructions page lacked critical information about how players win or lose (e.g., collect all bones to unlock the exit). This was not a bug per se but was a necessary addition for clarity.
4. Doug's Movement and Controls:
 - The arrow keys were initially non-functional (We wanted both, WASD and arrow keys to move Doug).
 - A reset function was missing. After losing the game and being presented with a "Game Over" screen, players had no way to restart.
 - After implementing the reset function, a new bug appeared: when restarting the game, the exit failed to spawn even after collecting all the required bones.
5. Missing Audio: The game lacked sound effects and music, which diminished the overall player experience.

By addressing these issues during the testing phase, we aimed to improve the game's stability, functionality, and overall user experience.

Unit Tests

During the testing phase, we focused on verifying individual components of the game in isolation to ensure their correctness and functionality. Below are the key features tested through unit tests:

Doug

- Singleton Pattern: Ensured Doug's instantiation adhered to the Singleton design pattern, maintaining a single instance throughout the game.
 - Within DougSingletonTest: testSingletonBehavior, testSingletonReset
- Movement and Boundary Clamping: Tested Doug's ability to move within the defined boundaries of the maze, preventing him from crossing into restricted areas.
 - Within DougTest: testMovement, testBoundaryClamping

Game Systems

Several core game functionalities were tested independently:

- Game Pause Test: Ensured the game could be paused and resumed without disrupting its state using the escape (esc) key.
 - Within GamePauseTest: testTogglePause
- Game Timer Test: Tested the timer's ability to start and stop. Verified that the timer stopped the game after a given duration.
 - Within GameTimerTest: testTimerStarts, testTimerStops, testFixedDuration

Game Objects:

- Bush Test: Verified bush object creation and rendering with valid and null images. Confirmed the bush bounding dimensions for collision detection.
 - Within BushTest: testConstructorValidResource, testRenderWithValidImage, testRenderWithNullImage, testGetBounds
- Steak Test: Verified the functionality of steaks, including how they interact with Doug and the effect on gameplay (e.g., appearing and disappearing on a timer).
 - Within SteakTest: testSteakSetAliveTrue, testSteakSetAliveFalse, testSteakTick

Other Features:

- Winning Screen Test: Confirmed the winning screen displayed accurately when victory conditions were met.
 - Within WinningScreenTest: testRender
- Health and Score Test: Validated that health and score were updated correctly during gameplay.
 - Within HealthScoreTest: testHealthDecrease, testScoreIncrease
- Sound Effects: Verified proper playback of proper/valid sound files for our game. Also ensured the game handled invalid or missing sound files properly.
 - Within SoundEffectTest: testPlayValidFile, testPlayInvalidFile, testPlayIOException, testPlayUnsupportedAudioFile

Integration Tests

Additionally, we focused on verifying interactions between various components of our game to ensure they work together well. We have implemented integration tests to validate the coordination between logic, user interface, game objects, and core systems. Below are the key interactions tested through integration tests:

Doug's Test:

Collision; Verified Doug's interactions with various game entities:

- Enemies: Ensured that collisions with enemies correctly reduced Doug's health.
- Objects: Checked that collisions with objects altered the score appropriately, based on the type of object (e.g., positive for collectibles like bones, negative for hazards).
 - Within DougCollisionTest.java: testCollisionWithEnemy, testCollisionWithReward, testCollisionWithPunishment

Doug Key Input and Movement in Game:

- Verified responsiveness of Doug to user inputs (WASD and arrow keys) and ensure movement stops while game is paused
 - Within DougKeyInputTest.java

Exit Tests:

- Tested whether the exit door remained hidden until all bone objects were collected.
- Verified that collecting the required bones "unlocked" the door, allowing Doug to access it.
 - Within ExitTest: testHidden, testActivated, testActivatedNoBone

User Interface:

Game Won Test:

- Simulated a collision between Doug and the unlocked exit point to ensure the game transitioned to the winning screen correctly when all victory conditions were met. Also verify the option to exit or restart the game works.
 - Within GameWonTest: testGameWon

Game Over Test:

- Manipulated Doug's health to validate the game over state:
- Confirmed that the game entered "Game Over" mode when Doug's health reached zero.
- Ensured that the game did not transition to "Game Over" mode if Doug's health was above zero, even when reduced significantly (e.g., to 1%).
 - Within GameOverTest: testGameOver, testGameNotOver

Main Menu Test:

- Tested interactions between the user and the main menu system, including navigation, menu rendering, and starting the game.
 - Within MainMenuTest: testNavigation, testRendering

Test Quality and Coverage Evaluation

Ensuring the quality of tests and achieving high code coverage are integral parts of testing. To achieve this, we focused on writing proper well formatted tests, identifying gaps in coverage, noting/addressing test failures, and refactoring sections of our code when needed. After implementing various kinds of tests for the various features and interactions mentioned earlier in the report, there were instances of low coverage in many classes that needed to be addressed. Below, we discuss some key aspects of our approach for specific components and the measures taken to ensure the quality of our test cases.

KeyInput and movement:

The KeyInput class, for example, underwent significant improvement. Initially at very low coverage at 11%, key features like movement logic, pause functionality, and diagonal movement prevention

were untested. Refactoring velocity updates, simplified key state tracking with a keyStates map, and introduced a lastMovementKey to prevent diagonal movement. These changes reduced redundancy and enhanced testability. Expanded tests covered movement with WASD and arrow keys, pause toggling with VK_ESCAPE, and diagonal movement prevention. These efforts boosted coverage to roughly 95%, demonstrating the impact of structured improvements.

Sound Effect Test Coverage:

Testing the SoundEffect class aimed to ensure functionality and improve its test coverage from an initial 64% to 88%. Initially, the class lacked tests for handling invalid or missing files and triggering exceptions. Unit tests were designed to cover scenarios, including valid file playback, invalid or missing files, unsupported formats, and exception handling. For example, valid .wav files like bark.wav were verified to play successfully, while nonexistent files (e.g., nonexistent.wav) exited without disrupting the game. Corrupted or unsupported files (e.g., unsupported.txt) were tested to ensure exceptions like UnsupportedAudioFileException were handled properly, and restricted files (e.g., restricted.wav) validated the program's robustness under failure conditions. Additionally, System.err was redirected to capture log output for verification. As a result, test coverage increased to 88%

Bush/Obstacle Testing:

Testing for the Bush and Obstacle classes prioritized core functionality while achieving significant coverage improvements. The Bush class increased from 86% to 87%, while the Obstacle class jumped from 67% to 100%, largely due to shared inheritance. Tests for Bush indirectly validated Obstacle's functionality, such as the getType() method, eliminating the need for redundant unit tests. For Bush, tests confirmed proper initialization and loading of the bush.png resource, validated rendering behavior for both valid and null images, and ensured accurate collision boundary dimensions from getBounds(). By leveraging shared inheritance, we achieved robust coverage efficiently, demonstrating the value of focusing on realistic scenarios and avoiding redundant tests.

Main Menu Testing

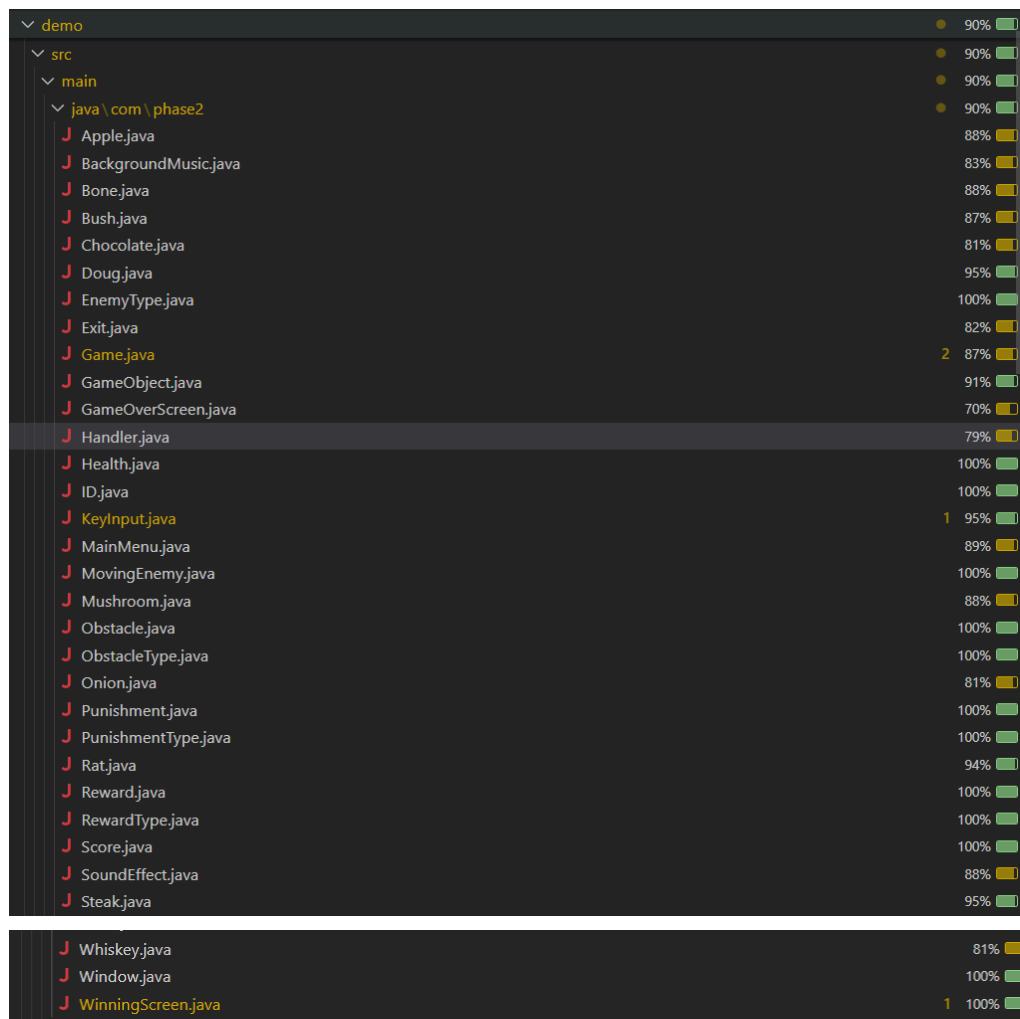
Initially, testing for the Main Menu was focused only on basic functionality, achieving 55% test coverage. The initial tests included only validating whether the game transitioned from a paused state to an active state when in the main menu the start button was selected and the "enter" key was pressed. While this test provided a foundation for verifying the integration between the main menu and the game system, there were many other functionalities that were untested. In order to improve its coverage and ensure the robustness of the Main Menu, other tests were added, among these the navigation test, analyzing the response of the keybinds, the interaction with other main menu options such as seeing the instructions or exiting the game and finally the rendering test, where we used a mock graphics object to validate that key rendering methods such as drawing strings, setting colors, and fonts were invoked appropriately when displaying the instructions in the screen. By expanding the scope of the tests, the Main Menu coverage increased to the point of reaching 86% thereby improving the reliability and functionality of the Main Menu.

Exit testing:

When testing for Exit most of its functionality were already shared with other classes. Tests such as the object being safely created and added into the game was already based around testing for similarly extended classes. The core functionality of the exit that needed to be tested would be its ability to hide from the player until the number of bones had been picked up. To test this, we looked for a variable that decided whether or not the exit should appear. A review of that section of the code revealed that

possible weaknesses would be that the visibility of the exit point is based entirely on checking whether boneScore has reached or surpassed boneTotal and rendering it without a set variable to define its status, making it difficult to apply functionality such as a sound cue when set all bones has been collected. Fixing this we added the hidden boolean which keeps track of whether or not Exit should still be hidden or interactable.

Ultimately, our project achieved a coverage of 90% during this testing phase of our game, seen in the picture below.



Findings and Missed Features/Interactions

The testing phase provided valuable insights into the game's strengths and areas for improvement. While many features were thoroughly validated, some may have been missed, highlighting the potential for further refinement. A key finding was how certain classes would indirectly validate the functionality of other classes, which may have eliminated redundant tests (for instance, the examples with Bush/Obstacle, and Exit mentioned earlier). Additionally, integration tests, like those involving Doug interacting with rewards, punishments, and enemies helped ensure the game mechanics worked smoothly.

Some possible missed features in our testing process includes exception handling, such as the IOException in the Bush constructor, which was not tested as it rarely occurs during gameplay. This is

also true for various other classes within our project, notably many of the game objects (like punishments, rewards, etc). Additionally, we did not perform integration tests for sound effects being played during specific events, such as collisions with rewards, punishments, or enemies like rats. Beyond these, other features or interactions may have been missed which could further validate the game's functionality, and improve our coverage.

Challenges and Things for the Future

While high coverage was achieved, challenges have been faced leaving us thinking of ways we could improve for the future. Possible steps we can take for the future is handling the possibilities our code could fail in ways we didn't expect. These failures could be handled so that the current progress the player has reached won't be lost if future levels are implemented. Additionally, while most components reached over 80% coverage, achieving full coverage was challenging, especially for rarely triggered paths like exception handling. Testing complex interactions, like simultaneous collisions or Doug interacting with multiple objects, also required significant setup and debugging, adding to the difficulty. Expanding integration tests could focus on validating interactions like sound effects during collisions with rewards, punishments, or enemies. This would ensure more comprehensive coverage of the game's dynamic features and improve overall system reliability.

Conclusion

Finally, the testing phase allowed us to improve the stability, functionality and overall quality of our game. By addressing previously identified bugs and expanding both unit and testing coverage, we were able to ensure that key features worked as intended. While our testing efforts made us achieve a high game coverage, the process also made us understand the importance of anticipating unexpected failures and designing a system to handle them gracefully, which is a valuable lesson that can be applicable for future projects.